

Plan de Proyecto — Motion ML

Software de captura, entrenamiento y exportación de movimientos para Unity

Contexto del Problema

Se está desarrollando un software en Unity que, mediante **MediaPipe** y **DollarsMono**, captura el movimiento de una persona y lo replica en un modelo 3D. El nuevo requerimiento es implementar un sistema de **Machine Learning** que permita:

- Decirle al modelo "realiza el movimiento X" (ej. Macarena) y que lo ejecute.
- Decirle "aprende este movimiento", que replique lo que el usuario hace y lo guarde para reproducirlo después.

Solución Propuesta

Separar el problema en dos partes:

1. **Software externo (Python)**: captura movimientos desde video, los entrena con ML y exporta el modelo.
2. **Unity**: recibe el modelo exportado y lo aplica al modelo 3D existente.

Modelo 3D — Información Clave

El modelo 3D usado en Unity ("El bueno") tiene un rig de **Mixamo**, identificado por los huesos con prefijo `mixamorig:`:

```
Armature.001
├── mixamorig:Hips
│   ├── mixamorig:LeftLeg
│   ├── mixamorig:LeftFoot
│   ├── mixamorig:Spine1
│   ├── mixamorig:LeftArm
│   └── mixamorig:LeftHand
└── ...
```

Esto es ventajoso porque Mixamo tiene huesos estandarizados y el mapeo con MediaPipe está bien documentado por la comunidad.

Stack Tecnológico

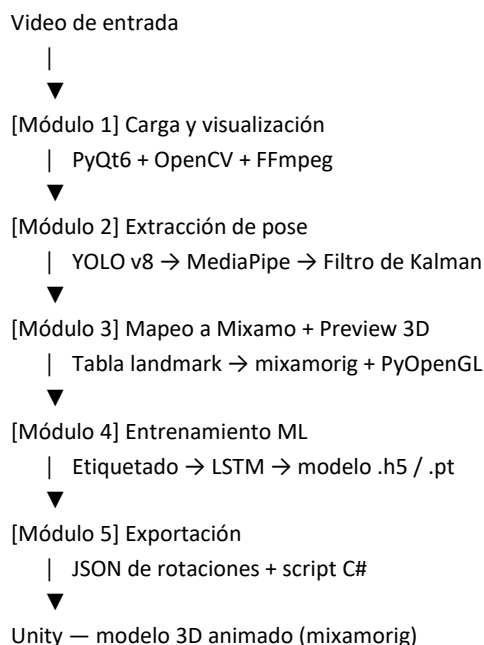
Capa	Tecnología	Propósito
Interfaz	PyQt6	Ventana principal, reproductor de video, paneles
Video	OpenCV + FFmpeg	Carga, reproducción y extracción de frames
Detección	YOLO v8	Detectar y aislar a la persona del fondo
Pose	MediaPipe Pose	Extraer 33 landmarks del cuerpo
Suavizado	Filtro de Kalman	Eliminar temblores entre frames
Vista previa	PyOpenGL	Preview 3D ligero del movimiento
ML	LSTM (TensorFlow / PyTorch)	Aprender y predecir secuencias de movimiento

Capa	Tecnología	Propósito
Exportación	JSON + C#	Comunicar rotaciones de huesos a Unity
Unity	Script C#	Leer JSON y animar el mixamorig

Por qué YOLO + MediaPipe

- **YOLO v8** detecta y recorta a la persona del fondo antes de pasar el frame a MediaPipe. Mejora el desempeño en videos con ruido visual o múltiples personas.
- **MediaPipe Pose** extrae los 33 landmarks estandarizados del cuerpo con alta precisión.
- Juntos forman un pipeline robusto: YOLO detecta → MediaPipe extrae → Kalman suaviza.

Pipeline Completo



Módulos del Software

Módulo 1 — Carga y Visualización de Video

Responsabilidad: Interfaz principal donde el usuario carga un video y lo visualiza.

Funcionalidades:

- Cargar video desde disco (MP4, AVI, MOV)
- Reproductor con controles (play, pause, frame a frame)
- Selector de frames para marcar inicio/fin de un movimiento
- Panel lateral para etiquetar el movimiento con un nombre

Tecnologías: PyQt6, OpenCV, FFmpeg

Módulo 2 — Extracción de Pose

Responsabilidad: Extraer los landmarks del cuerpo de cada frame del video.

Pipeline interno:

1. YOLO v8 detecta a la persona y genera un bounding box.
2. Se recorta el frame con el bounding box.
3. MediaPipe Pose extrae los 33 landmarks.
4. El Filtro de Kalman suaviza las trayectorias entre frames.

Salida: Array de landmarks [frame, landmark_id, x, y, z, visibility]

Tecnologías: Ultralytics YOLO, MediaPipe, filterpy (Kalman)

Módulo 3 — Mapeo a Mixamo + Preview 3D

Responsabilidad: Traducir los landmarks de MediaPipe a huesos del rig Mixamo y mostrar una vista previa.

Tabla de mapeo principal:

MediaPipe Landmark	Hueso Mixamo
LEFT_HIP	mixamorig:LeftUpLeg
LEFT_KNEE	mixamorig:LeftLeg
LEFT_ANKLE	mixamorig:LeftFoot
RIGHT_HIP	mixamorig:RightUpLeg
RIGHT_KNEE	mixamorig:RightLeg
RIGHT_ANKLE	mixamorig:RightFoot
LEFT_SHOULDER	mixamorig:LeftArm
LEFT_ELBOW	mixamorig:LeftForeArm
LEFT_WRIST	mixamorig:LeftHand
RIGHT_SHOULDER	mixamorig:RightArm
RIGHT_ELBOW	mixamorig:RightForeArm
RIGHT_WRIST	mixamorig:RightHand
NOSE	mixamorig:Head

Funcionalidades UI:

- Panel para activar/desactivar qué huesos capturar (manos, pies, torso, cabeza)
- Vista previa 3D en tiempo real mientras reproduce el video
- Validación visual antes de exportar

Tecnologías: PyOpenGL, NumPy

Módulo 4 — Entrenamiento ML

Responsabilidad: Aprender movimientos etiquetados y poder predecirlos por nombre.

Flujo de entrenamiento:

1. El usuario graba/carga varios videos del mismo movimiento y los etiqueta (ej. "Macarena").
2. Se construye un dataset de secuencias [nombre → array de frames de landmarks].
3. Se entrena un modelo **LSTM** que aprende la distribución temporal de cada movimiento.
4. El modelo se guarda como .h5 (TensorFlow) o .pt (PyTorch).

Flujo de predicción:

1. Unity (o el software) pide "ejecuta Macarena".
2. El predictor carga el modelo y genera la secuencia de rotaciones.
3. Se envía a Unity vía JSON.

Por qué LSTM: Los movimientos son secuencias temporales. El LSTM tiene memoria de pasos anteriores, lo que le permite aprender el ritmo y la continuidad del movimiento.

Tecnologías: TensorFlow / PyTorch, NumPy, scikit-learn

Módulo 5 — Exportación a Unity

Responsabilidad: Generar los archivos que Unity necesita para animar el modelo.

Archivos generados:

- poses.json — rotaciones de cada hueso por frame
- movements_db.json — base de datos de todos los movimientos aprendidos
- modelo.h5 O modelo.pt — modelo LSTM entrenado
- MixamoAnimator.cs — script C# listo para arrastrar al modelo en Unity

Estructura del JSON exportado:

```
{
  "movement_name": "Secuencia1"
  "fps": 30,
  "frames": [
    {
      "frame": 0,
      "bones": {
        "mixamorig:LeftArm": { "x": 0.12, "y": -0.34, "z": 0.56 },
        "mixamorig:RightArm": { "x": -0.10, "y": -0.30, "z": 0.50 }
      }
    }
  ]
}
```

Tecnologías: JSON, C# (Unity), Python

Estructura de Carpetas

```
motion_ml/
|
├─ ui/                # Interfaz principal — PyQt6
|   ├── main_window.py    # Ventana principal + reproductor de video
|   ├── bone_selector.py  # UI para activar/desactivar huesos Mixamo
|   ├── preview_3d.py     # Vista previa del modelo 3D — PyOpenGL
|   └─ label_panel.py     # Panel para etiquetar movimientos
|
├─ pose_extraction/    # YOLO + MediaPipe + suavizado
|   ├── detector.py       # YOLO v8 — detección y recorte de persona
|   ├── landmark_extractor.py # MediaPipe Pose — extrae 33 landmarks
|   ├── smoother.py      # Filtro de Kalman — suaviza trayectorias
|   └─ mixamo_mapper.py   # Mapeo landmark → hueso mixamorig
|
├─ ml/                # Entrenamiento y predicción — LSTM
|   ├── dataset_builder.py # Construye dataset desde secuencias capturadas
|   ├── model.py          # Arquitectura LSTM — TensorFlow / PyTorch
|   ├── trainer.py        # Loop de entrenamiento + métricas
|   └─ predictor.py       # Inferencia: dado nombre → retorna secuencia
|
├─ export/            # Generación de archivos para Unity
|   ├── json_exporter.py  # Genera poses.json con rotaciones por frame
|   ├── unity_bridge.py   # Empaqueta modelo ML + JSON para Unity
|   └─ unity/
|       └─ MixamoAnimator.cs # Script C# que lee JSON y mueve los huesos
|
├─ data/              # Datos generados
|   ├── videos/          # Videos cargados por el usuario
|   ├── sequences/        # Secuencias de landmarks etiquetadas
|   ├── models/           # Modelos LSTM entrenados (.h5 / .pt)
|   └─ exports/           # JSONs listos para importar en Unity
|
├─ main.py            # Punto de entrada de la aplicación
├─ config.yaml        # Configuración global
└─ requirements.txt    # Dependencias Python
```

Dependencias Python (requirements.txt)

```
pyqt6
opencv-python
ffmpeg-python
ultralytics    # YOLO v8
mediapipe
filterpy       # Filtro de Kalman
PyOpenGL
PyOpenGL_accelerate
numpy
tensorflow     # o torch si se prefiere PyTorch
scikit-learn
```

Próximos Pasos

1. Configurar el entorno virtual e instalar dependencias.
2. Programar **Módulo 1** — ventana principal con reproductor de video.
3. Programar **Módulo 2** — pipeline YOLO + MediaPipe + Kalman.
4. Programar **Módulo 3** — mapeo Mixamo + preview 3D.
5. Programar **Módulo 4** — dataset, entrenamiento LSTM y predicción.
6. Programar **Módulo 5** — exportación JSON + script C# para Unity.
7. Integración y pruebas end-to-end con el modelo "El bueno".